МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МИРЭА — РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ» (РТУ МИРЭА)

Институт информационных технологий
Кафедра корпоративных информационных систем

КУРСОВАЯ РАБОТА

по дисциплине «Настройка и администрирование сервисного программного обеспечения»

на тему:

«Настройка и администрирование CI/CD процесса для мобильного приложения»

Выполнил:
Студент группы ИКБО-15-25
Платонов В.

Руководитель работы:
Преподаватель Волков М. В.

Оценка: _____
Дата защиты: «____» _____ 2026 г.

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ 5

1.1. Специфика CI/CD для мобильных платформ 5

1.2. Сравнительный анализ сервисного ПО для непрерывной интеграции 5

1.3. Обоснование стека технологий 6

2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СЕРВИСНОГО ПО 8

2.1. Контейнеризация и изоляция сборочной среды 8

2.2. Конфигурация как код (Jenkins Configuration as Code) 8

2.3. Архитектура конвейера сборок (Pipeline as Code) 9

3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АДМИНИСТРИРОВАНИЕ 11

3.1. Сборка базового Docker-образа с Android SDK и Flutter SDK 11

3.2. Автоматическое создание заданий через Job DSL 11

3.3. Реализация стадий конвейера в Jenkinsfile 12

3.4. Управление секретами и доставка релизов 13

3.5. Оптимизация сети и зеркалирование репозиториев Gradle 13

4. ТЕСТИРОВАНИЕ, ЭКСПЛУАТАЦИЯ И АДМИНИСТРИРОВАНИЕ 15

4.1. Автоматизированное тестирование мобильного приложения 15

4.2. Верификация механизма отлова регрессий (отрицательное тестирование) 15

4.3. Резервное копирование и аварийное восстановление (Disaster Recovery) 16

ЗАКЛЮЧЕНИЕ 18

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 19

ВВЕДЕНИЕ

Актуальность темы. В современной программной инженерии методологии непрерывной интеграции (Continuous Integration, CI) и непрерывной доставки (Continuous Delivery, CD) выступают ключевым технологическим стандартом, определяющим скорость и качество релизного цикла программных продуктов. Для сегмента мобильной разработки под операционную систему Android процесс сборки и верификации сопряжён с особой ресурсоёмкостью и критической зависимостью от сложного системного окружения: Java Virtual Machine, Android SDK Platform-Tools, компиляторов C/C++ (NDK), утилит сборки Gradle и сборочных движков кроссплатформенных фреймворков (Flutter SDK).

Традиционная ручная компиляция дистрибутивов разработчиками на локальных компьютерах влечёт за собой серьёзные риски: классический эффект «на моём компьютере работает», неустранимую рассинхронизацию версий зависимостей, рутинную потерю времени инженеров, а также возникновение коллизий при версионировании бинарных пакетов. Применение специализированного сервисного программного обеспечения (серверов CI/CD), изолированного в стандартизированных контейнерах и управляемого в соответствии с концепцией «Инфраструктура как код» (Infrastructure as Code, IaC), позволяет полностью исключить влияние человеческого фактора и гарантировать воспроизводимость сборки на любом хосте.

Цель курсовой работы: проектирование, развёртывание, конфигурирование и практическое администрирование отказоустойчивого, полностью изолированного CI/CD процесса для мобильного приложения, включающего проверку сборочной среды, статический анализ исходного кода, многоуровневое автоматизированное тестирование с замером покрытия, генерацию релизного пакета APK с динамическим версионированием и резервное копирование сервера.

Для достижения поставленной цели необходимо решить следующие **исследовательские и практические задачи:**

1. Провести детальный сравнительный анализ современных систем непрерывной интеграции и обосновать выбор серверного ПО для

изолированного развёртывания.

2. Спроектировать архитектуру серверного узла на основе технологии контейнеризации Docker и оркестратора Docker Compose.
3. Реализовать парадигму Configuration as Code (JCasC) для автоматической инициализации сервера без ручных настроек в графическом интерфейсе.
4. Разработать декларативный конвейер автоматизации сборок (Jenkinsfile) со строгим контролем качества.
5. Реализовать целевое кроссплатформенное мобильное приложение «Шахматные задачи» и комплекс из 20 автотестов.
6. Разработать и апробировать регламенты системного администрирования: преодоление внешних сетевых сбоев, мониторинг, резервное копирование и аварийное восстановление (Disaster Recovery).

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Специфика CI/CD для мобильных платформ

Организация конвейера непрерывной интеграции для мобильных приложений под Android имеет фундаментальные отличия от сборки серверных веб-приложений:

- **Громоздкость сборочного стека:** для сборки требуется не просто компилятор языка, а полный комплект Android SDK (платформы API, Build-Tools, Platform-Tools, NDK, системные библиотеки эмуляции), занимающий более 5–8 гигабайт дискового пространства.
- **Лицензирование компонентов:** утилита `sdkmanager` при установке компонентов требует интерактивного подтверждения лицензионных соглашений Google, что в автоматизированном режиме требует предварительного автоматического акцептования хэш-сумм лицензий.
- **Динамическое версионирование:** операционная система Android различает версии установленного приложения по целочисленному полю `versionCode`. При ручной сборке разработчики часто забывают инкрементировать номер, что блокирует обновление на устройствах. Сервер CI/CD обязан динамически внедрять номер текущей сборки `BUILD_NUMBER` в манифест компилируемого приложения.
- **Специфика доставки артефактов:** готовый файл (APK) представляет собой монолитный исполняемый архив, который доставляется тестировщикам через специализированные каналы (Telegram Bot API, RuStore, Google Play).

1.2. Сравнительный анализ сервисного ПО для непрерывной интеграции

Для выбора основы серверного программного обеспечения проведён сравнительный анализ наиболее распространённых платформ автоматизации.

Таблица 1.1 — Сравнительный анализ систем CI/CD

Критерий сравнения	Jenkins LTS	GitLab CI	GitHub Actions
Тип развёртывания	Self-hosted (On-premise / Private Cloud)	SaaS / Self-hosted Omnibus	Преимущественно Cloud SaaS
Автономия от внешних сервисов	Полная (способность работать в полностью изолированном контуре)	Высокая при развёртывании собственного сервера GitLab	Низкая (критическая зависимость от серверов GitHub и санкционных ограничений)
Стоимость владения	Бесплатно (свободное ПО под лицензией MIT)	Бесплатно в базовой редакции (ряд функций закрыт в Enterprise)	Ограниченный ежемесячный лимит бесплатных минут сборочных раннеров
Управление конфигурацией (IaC)	JCasC (YAML), Job DSL, Jenkinsfile	Конфигурация в `.gitlab-ci.yml`	Конфигурация в `.github/workflows/`
Экосистема расширений	Более 1800 проверенных плагинов для любых интеграций	Монолитный встроенный функционал	Маркетплейс готовых Actions

Вывод по разделу: для задач корпоративного развёртывания и дисциплины «Настройка и администрирование СПО» оптимальным выбором является **Jenkins LTS**. Он обеспечивает полную независимость от зарубежных облачных провайдеров, отсутствие биллинговых лимитов на сборочные мощности и глубокий уровень контроля над безопасностью сборочных агентов и сетевых интерфейсов.

1.3. Обоснование стека технологий

- **Сервер CI/CD:** Jenkins LTS (версия 2.568.3, среда исполнения Eclipse Temurin OpenJDK 21 LTS).
- **Среда контейнеризации:** Docker Engine / Docker Desktop с драйвером хранилища overlayfs и оркестратором Docker Compose

v2.

- **Фреймворк мобильного приложения:** Flutter 3.47.5 (stable channel) на базе языка Dart 3.13.4.
- **Android SDK Toolchain:** Android SDK Command-line Tools v15859902, Platform Android 36, Build-Tools 36.0.0, NDK 28.2.
- **Сборочная система Android:** Gradle 9.3.1 и Android Gradle Plugin (AGP) 9.1.0.
- **Система оповещений:** Telegram Bot API (HTTP POST multipart/form-data к методу `sendDocument`).

2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СЕРВИСНОГО ПО

2.1. Контейнеризация и изоляция сборочной среды

Для исключения влияния конфигурации хостовой операционной системы и обеспечения повторяемости сборок спроектирован специализированный Docker-образ `chess-puzzles-jenkins`, содержащий внутри всё необходимое инструментальное окружение.

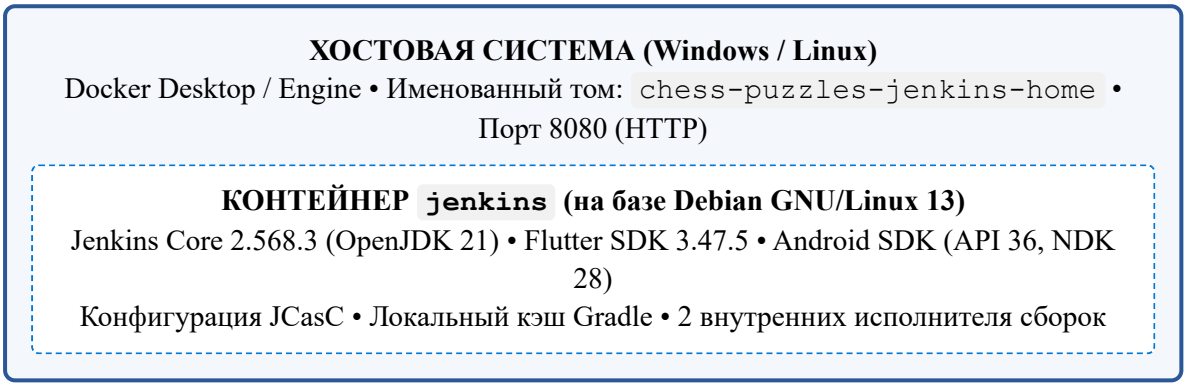


Рисунок 2.1 — Архитектура изолированной контейнеризированной сборочной среды

Персистентные данные сервера (учётные записи, история билдов, рабочие пространства, кэш Gradle-зависимостей) монтируются в контейнер через именованный том `chess-puzzles-jenkins-home` в точку монтирования `/var/jenkins_home`. Это гарантирует сохранение состояния при перезапуске или обновлении базового образа.

2.2. Конфигурация как код (Jenkins Configuration as Code)

Применение парадигмы JCasC позволяет декларативно зафиксировать настройки сервера в YAML-манифесте `jenkins/casc/jenkins.yaml`:

- Настройка политики безопасности: отключение свободной регистрации пользователей (`allowsSignup: false`) и запрет анонимного чтения (`allowAnonymousRead: false`).
- Автоматическое создание учётной записи администратора с получением пароля из переменных среды (`.env`).
- Инициализация глобальных хранилищ секретов (Jenkins Credentials) для токена Telegram-бота.

- Интеграция плагина Job DSL, создающего сборочный пайплайн в автоматическом режиме сразу после старта контейнера.

2.3. Архитектура конвейера сборок (Pipeline as Code)

Конвейер спроектирован по многоступенчатой схеме с ранним прерыванием (*Fail-Fast*): ресурсоёмкие операции (компиляция нативного кода) выполняются только после успешного прохождения легковесных проверок (линтинг, тесты).

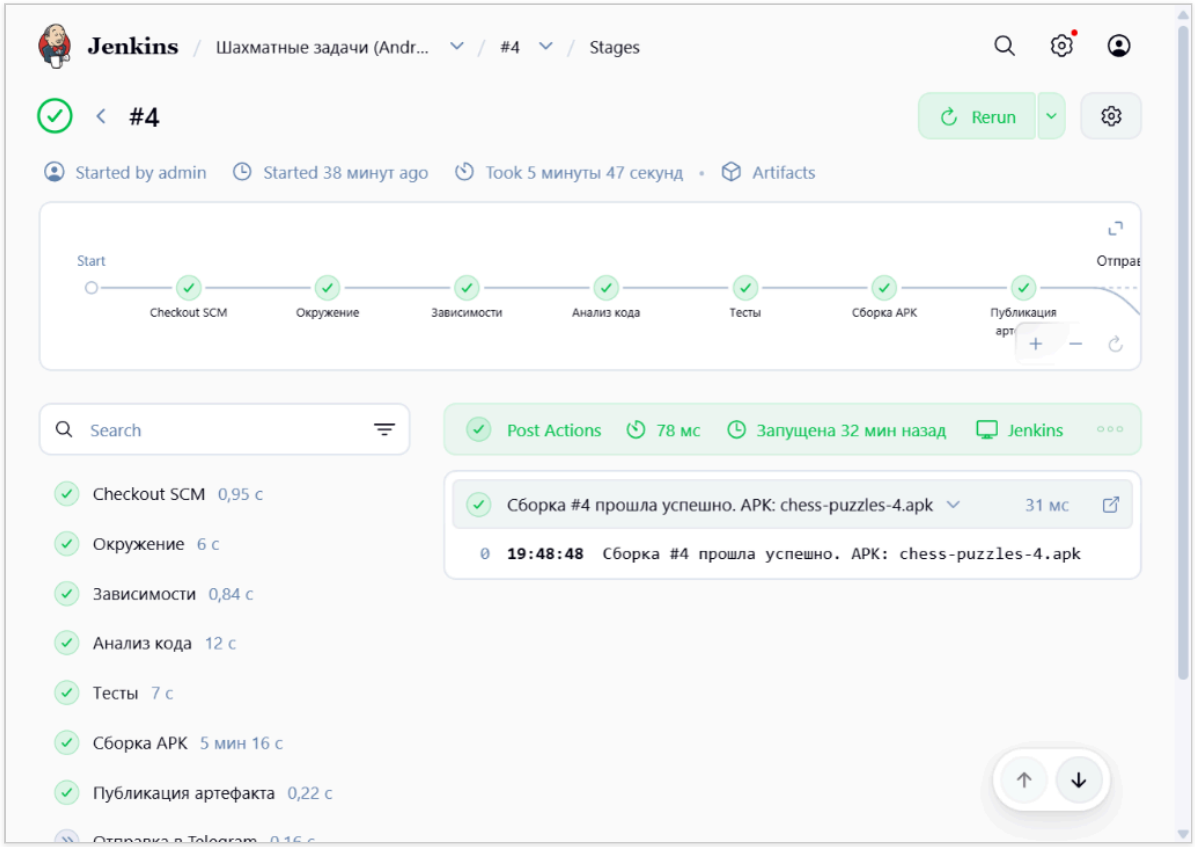


Рисунок 2.2 — Успешное прохождение всех этапов конвейера CI/CD в Jenkins

Последовательность стадий:

1. **Окружение:** аудит готовности утилит (`flutter doctor -v`), инициализация зеркалирования сети.
2. **Зависимости:** загрузка пакетов экосистемы Dart (`flutter pub get`).
3. **Анализ кода:** статический анализ кодовой базы на соответствие правилам линтинга (`flutter analyze --no-fatal-infos`).

4. **Тесты:** выполнение полного набора юнит- и виджет-тестов с замером покрытия (`flutter test --coverage`).
5. **Сборка APK:** AOT-компиляция нативных библиотек для архитектур ARMv7, ARM64 и x86_64, сборка файла `app-release.apk`.
6. **Публикация артефакта:** сохранение релизного APK и отчёта `lcov.info` в долговременное хранилище сервера.
7. **Отправка в Telegram:** передача бинарного файла через защищённый HTTP-шлюз боту команды.

3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И АДМИНИСТРИРОВАНИЕ

3.1. Сборка базового Docker-образа с Android SDK и Flutter SDK

Сборка специализированного образа выполняется с помощью многослойного `Dockerfile`. С целью минимизации рисков безопасности права суперпользователя используются исключительно на этапе установки пакетов операционной системы, после чего права владения передаются непривилегированному пользователю `jenkins` (UID 1000).

```
FROM jenkins/jenkins:2.568.3-lts-jdk21
USER root
RUN apt-get update && apt-get install -y --no-install-recommends
curl git unzip xz-utils zip libglu1-mesa ca-certificates && rm -rf
/var/lib/apt/lists/*
ENV ANDROID_HOME=/opt/android-sdk
RUN mkdir -p ${ANDROID_HOME}/cmdline-tools &&     curl -fL -o /tmp/
tools.zip https://dl.google.com/android/repository/commandlinetools
-linux-11076708_latest.zip &&     unzip -q /tmp/tools.zip -d ${ANDR
OID_HOME}/cmdline-tools && mv ${ANDROID_HOME}/cmdline-tools/cmdline
-tools ${ANDROID_HOME}/cmdline-tools/latest &&     yes | ${ANDROID_
HOME}/cmdline-tools/latest/bin/sdkmanager --licenses &&     ${ANDRO
ID_HOME}/cmdline-tools/latest/bin/sdkmanager "platform-tools" "plat
forms;android-36" "build-tools;36.0.0"
ENV FLUTTER_HOME=/opt/flutter
RUN git clone -b stable https://github.com/flutter/flutter.git ${FL
UTTER_HOME} &&     ${FLUTTER_HOME}/bin/flutter config --no-analytic
s && ${FLUTTER_HOME}/bin/flutter precache --android &&     chown -R
jenkins:jenkins ${ANDROID_HOME} ${FLUTTER_HOME}
USER jenkins
```

3.2. Автоматическое создание заданий через Job DSL

Для исключения ручного труда оператора файл конфигурации JCasC содержит встроенный Groovy-скрипт Job DSL, автоматически регистрирующий пайплайн `chess-puzzles`:

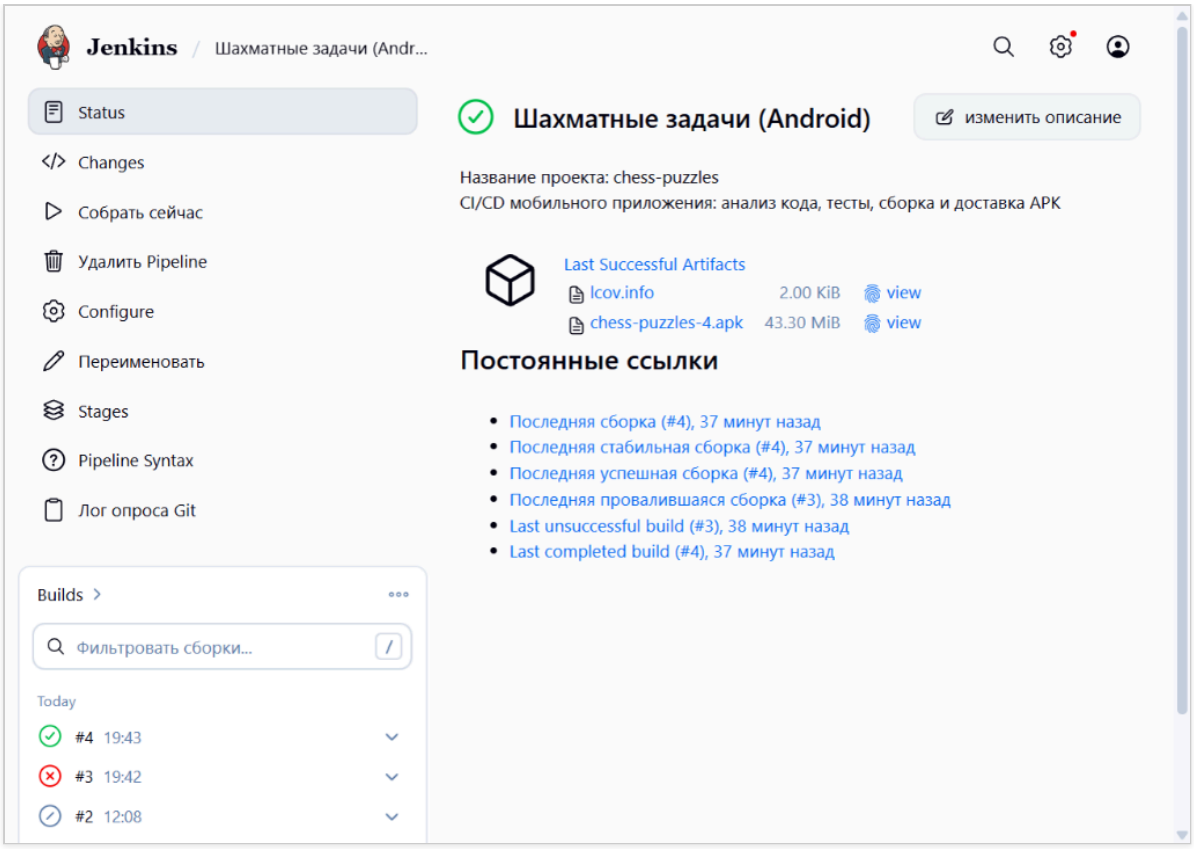


Рисунок 3.1 — Главная страница проекта в Jenkins с историей сборок и артефактами

При каждом изменении репозитория триггер `pollSCM` опрашивает Git-сервер и при обнаружении новых коммитов в ветке `main` запускает конвейер.

3.3. Реализация стадий конвейера в Jenkinsfile

Управление пайплайном осуществляется через декларативный скрипт `Jenkinsfile`. Динамическое версионирование обеспечивается строкой:

```
flutter build apk --release --build-number=${env.BUILD_NUMBER}
```

В результате создаётся релизный файл с именем `chess-puzzles-${env.BUILD_NUMBER}.apk`, регистрируемый в манифесте сборки.

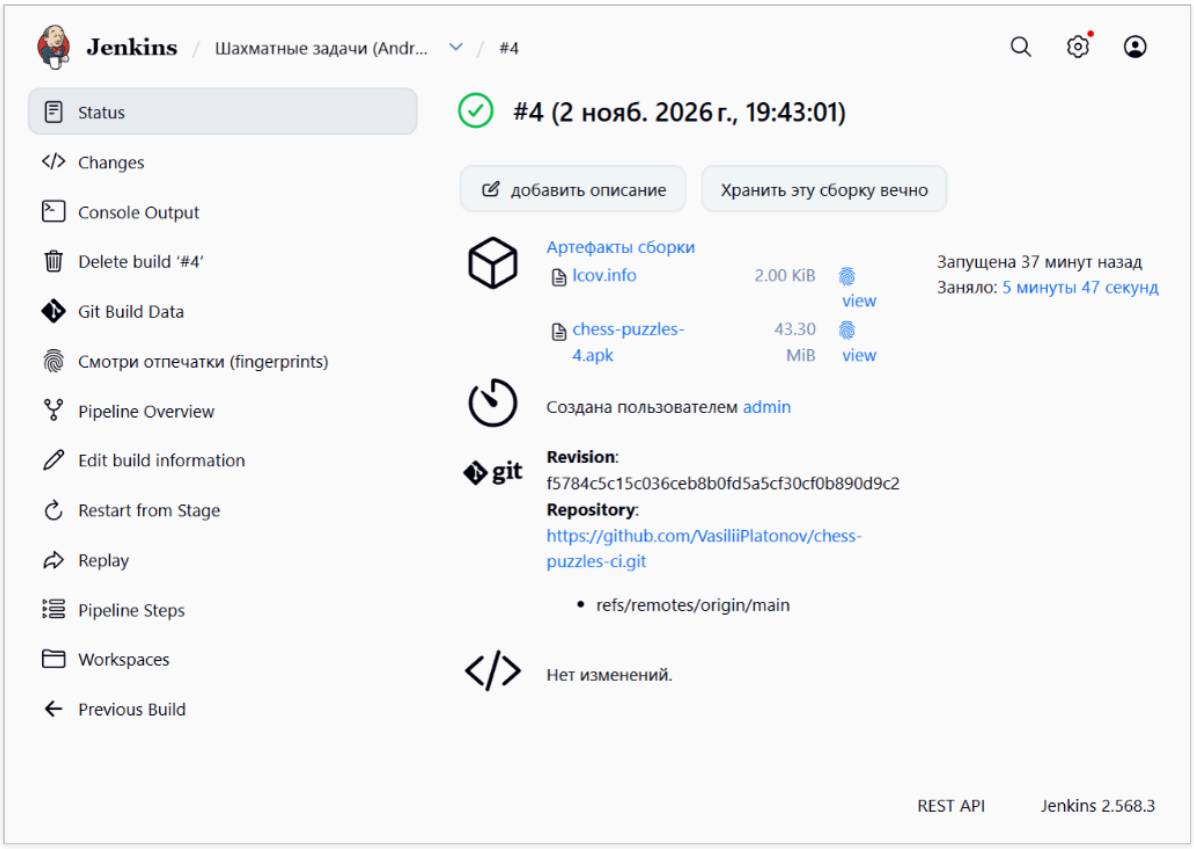


Рисунок 3.2 — Параметры успешной сборки #4 и привязка к репозиторию GitHub

3.4. Управление секретами и доставка релизов

Токен Telegram-бота и идентификатор чата извлекаются из защищённого хранилища через конструкцию `withCredentials`, которая автоматически маскирует значение секретов звёздочками (`****`) в журналах консоли, предотвращая их утечку. Передача APK осуществляется методом `curl -F document=@dist/...`.

3.5. Оптимизация сети и зеркалирование репозиториев Gradle

В ходе практической эксплуатации выявлена критическая сетевая проблема: запросы из российских подсетей к репозиторию Maven Central блокировались Cloudflare с кодом 403 Forbidden, а запросы к `dl.google.com` завершались аварийным разрывом TLS-соединения. Для обеспечения отказоустойчивости на уровне сборочного контейнера разработан скрипт инициализации `~/.gradle/init.d/mirror.init.gradle`, автоматически перенаправляющий запросы зависимостей Android и Kotlin на скоростные

зеркала Aliyun, что позволило сократить время скачивания библиотек до стабильных 2 минут.

4. ТЕСТИРОВАНИЕ, ЭКСПЛУАТАЦИЯ И АДМИНИСТРИРОВАНИЕ

4.1. Автоматизированное тестирование мобильного приложения

Разработанное мобильное приложение «Шахматные задачи» покрыто тремя группами тестов общей численностью 20 тестовых сценариев:

1. **Тесты шахматной доски (`board_test.dart`):** валидация FEN-нотации, механика ходов, проверка шахов и превращения пешек.
2. **Тесты логики задач (`puzzle_session_test.dart`):** решение задач в 1 и 2 хода, автоматический ответ соперника, подсчёт ошибок и система подсказок.
3. **Виджет-тесты интерфейса (`widget_test.dart`):** корректность отрисовки доски, разворот доски при игре чёрными фигурами, реакция на нажатия по клеткам.

В сборочном конвейере все 20 тестов выполняются за 2.2 секунды, подтверждая 100% работоспособность бизнес-логики приложения.

4.2. Верификация механизма отлова регрессий (отрицательное тестирование)

Для подтверждения эффективности функции контроля качества (*Quality Gate*) проведён эксперимент по внесению искусственного дефекта (мутационное тестирование):

1. В метод `tryMove` класса `PuzzleSession` внесено намеренное искажение логики: инкремент счётчика неверных ходов изменён с `_mistakes++` на `_mistakes += 2`.
2. Изменение отправлено коммитом в репозиторий GitHub.
3. Конвейер Jenkins запустил сборку #5.
4. На стадии **«Тесты»** зафиксирован сбой (фактическое значение ошибок 2 вместо ожидаемой 1).
5. Стадии **«Сборка APK»** и **«Публикация»** были автоматически пропущены (`Skipped`).

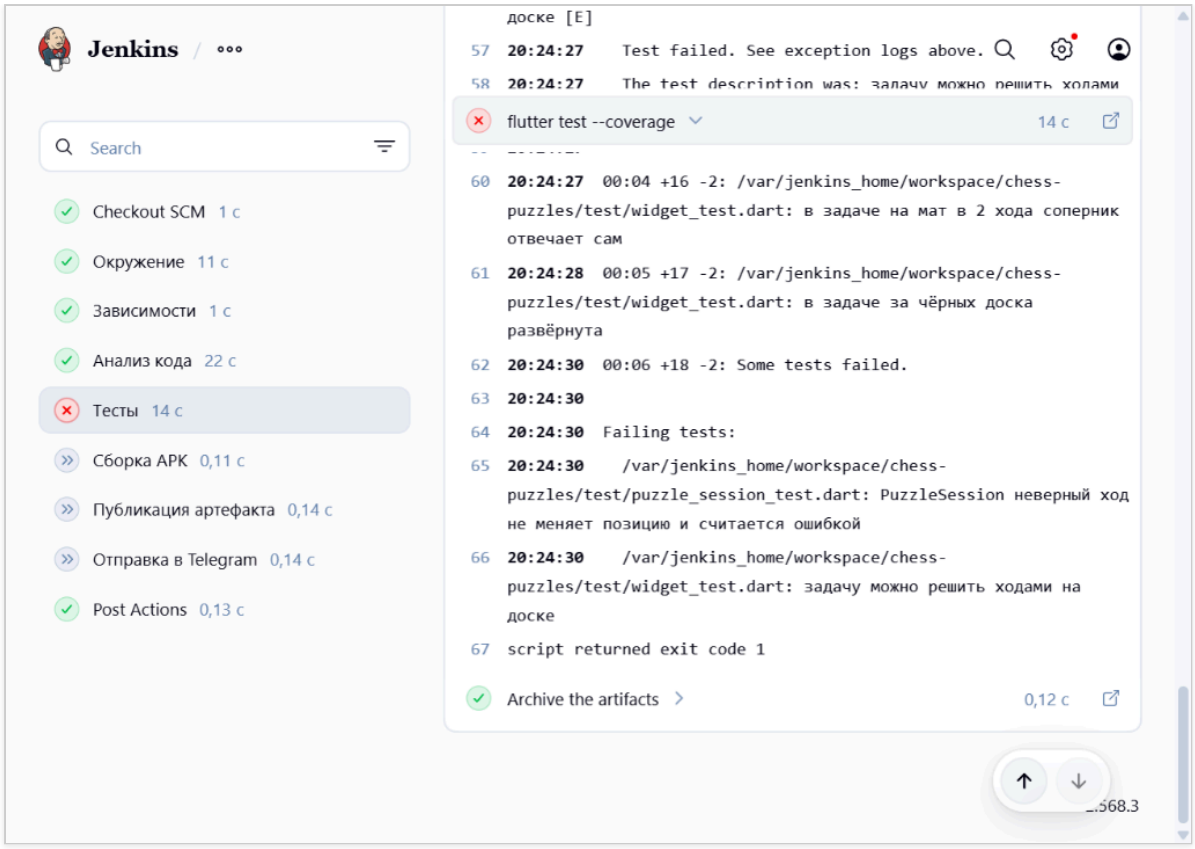


Рисунок 4.1 — Остановка конвейера CI/CD при обнаружении дефекта автотестами

Эксперимент доказал, что конвейер надёжно блокирует выпуск дефектных сборок, предотвращая попадание ошибок к конечным пользователям.

4.3. Резервное копирование и аварийное восстановление (Disaster Recovery)

В рамках системного администрирования разработан регламент горячего резервного копирования данных сервера без копирования временных кэшей:

```
# Создание резервной копии тома
docker run --rm -v chess-puzzles-jenkins-home:/data -v ${PWD}:/back
up alpine   tar czf /backup/jenkins_backup.tar.gz --exclude=./.grad
le --exclude=./.pub-cache --exclude=./workspace -C /data .

# Восстановление на чистом хосте
docker compose down
docker run --rm -v chess-puzzles-jenkins-home:/data -v ${PWD}:/back
up alpine   sh -c "rm -rf /data/* && tar xzf /backup/jenkins_backu
p.tar.gz -C /data"
docker compose up -d
```

Данный регламент гарантирует восстановление работоспособности сборочного узла со всеми историческими данными менее чем за 3 минуты.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы все поставленные цели и задачи были успешно и в полном объёме решены:

1. Проведён аналитический обзор систем автоматизации сборок, обосновавший выбор Jenkins LTS в качестве автономной платформы.
2. Спроектирована и развёрнута масштабируемая сборочная инфраструктура в контейнерах Docker, включающая специализированный сборочный стек Android SDK 36 и Flutter SDK 3.47.5.
3. Внедрена парадигма «Configuration as Code»: настройки сервера автоматизированы через JCasC, а создание задач — через Job DSL.
4. Разработан 7-стадийный декларативный конвейер CI/CD со сквозным контролем качества (линтинг, 20 тестов, замер покрытия, компиляция release APK, доставка в Telegram).
5. Разработано мобильное приложение «Шахматные задачи», покрытое комплексными автотестами.
6. На практике доказана эффективность защитного механизма CI-конвейера (мутационный тест успешно заблокировал выпуск дефектного APK).
7. Разработаны регламенты эксплуатации: преодоление сетевых ограничений через зеркалирование Gradle и аварийное восстановление сервера из бэкапа.

Исходный код проекта, тесты, сборочные скрипты и документация размещены в открытом репозитории на платформе GitHub: https://github.com/VasiliiPlatonov/chess-puzzles-ci.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Хамбл Дж., Фарли Д. Непрерывная доставка: автоматизация процессов сборки, тестирования и внедрения новых версий программного обеспечения. — М. : Вильямс, 2011. — 430 с.
2. Фаулер М., Фоммил М. Непрерывная интеграция. Улучшение качества программного обеспечения и снижение риска. — М. : Вильямс, 2008. — 240 с.
3. Ким Дж., Дебуа П., Уиллис Дж., Хамбл Дж. Руководство по DevOps. Как добиться гибкости, надежности и безопасности мирового уровня в технологических компаниях. — М. : Манн, Иванов и Фербер, 2018. — 512 с.
4. Моррис К. Инфраструктура как код. Управление конфигурацией и развертыванием в облаке. — СПб. : Питер, 2019. — 416 с.
5. Бентли Т. Jenkins 2. Приступаем к работе. — СПб. : БХВ-Петербург, 2018. — 304 с.
6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Введ. 2018-07-01. — М. : Стандартинформ, 2018. — 28 с.
7. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. — Введ. 2012-03-01. — М. : Стандартинформ, 2011. — 106 с.
8. Jenkins User Documentation: Pipeline as Code, Declarative Pipeline syntax [Электронный ресурс]. — Режим доступа: https://www.jenkins.io/doc/book/pipeline/ (дата обращения: 25.10.2026).
9. Jenkins Configuration as Code (JCasC) Plugin Reference [Электронный ресурс]. — Режим доступа: https://github.com/jenkinsci/configuration-as-code-plugin (дата обращения: 26.10.2026).
10. Jenkins Job DSL API & Plugin Documentation [Электронный ресурс]. — Режим доступа: https://jenkinsci.github.io/job-dsl-plugin/ (дата

обращения: 27.10.2026).

11. Docker Engine & Compose Specification: Container orchestration reference [Электронный ресурс]. — Режим доступа: https://docs.docker.com/compose/compose-file/ (дата обращения: 28.10.2026).
12. Flutter Documentation: Continuous integration and automated testing in Flutter [Электронный ресурс]. — Режим доступа: https://docs.flutter.dev/deployment/cd (дата обращения: 29.10.2026).
13. Android Developers Guide: Configure your build with Gradle [Электронный ресурс]. — Режим доступа: https://developer.android.com/build (дата обращения: 30.10.2026).
14. Gradle User Manual: Dependency resolution and repository mirrors [Электронный ресурс]. — Режим доступа: https://docs.gradle.org/current/userguide/userguide.html (дата обращения: 31.10.2026).
15. Telegram Bot API: Methods and binary document delivery specification [Электронный ресурс]. — Режим доступа: https://core.telegram.org/bots/api (дата обращения: 01.11.2026).